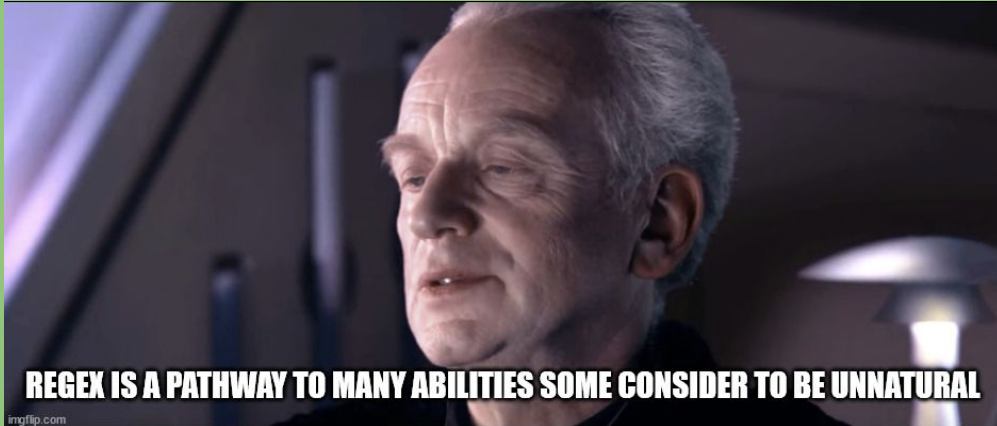


COMP2041

Week 2



**Thank you Kyu-Sang Kim for the
foundation of these slides!**

- **Welcome to Week 2 of COMP2041**
- Help Sessions start some time in late Week 3 or early Week 4
 - Schedule probably released in Week 3
- Week 1 & 2 labs due Week 3 Monday 12PM (midday) AEST
 - Lab due dates expected to normalise from Week 3

Overview

- Delimiters
- Useful UNIX Tools
- UNIX Pipelines “|”
- Tutorial Questions



Delimiters

- A delimiter is a specific character that separates fields in a line of text
- Common delimiters include: pipe (|), comma (,), colon (:), any amount of whitespace ()
- Tools like cut and sort capitalise on this, allowing us to specify what fields to manipulate
- Example of delimited data: **Nick|21|75|COMP2041**
 - 4 fields (name, age, WAM, course) separated by pipe (|).

- sort
- head/tail
- cut
- tr
- grep
- sed
- uniq
- wc
- join
- man

Photo of a new Linux user learning the command line



Cut command

- Extracts specific fields from each line
- **-d'<delimiter>**, sets the delimiter (default is **a tab**)
- **-f'<n>**, selects which field(s) to extract
 - **-f1** - extract the first field
 - **-f1,f3** - extract the first and third field
- Example of use: **cut -d'|' -f1 data.txt**
 - Extracts specifically the first field, on data that is delimited by a pipe '|'

Sort command

- Sorts lines of text alphabetically by default
 - Can even sort based on a specific field in delimited data
- `-t` sets the delimiter (same idea as `-d` in `cut`)
- `-k1<N>` selects which field to sort by
 - Typically, you would do something like `sort -k1`, to sort field 1, however we should do **sort -k1,1**.
- `-n` sorts numerically instead of alphabetically
- `-r` sorts in reverse order
- Example of use: **`cut -t'|' -k1 data.txt`**
 - Sorts based on the first field, on data that is delimited by a pipe `'|'`

Head / Tail command

head

- Outputs the first N lines of a file
- Default is 10 lines
- **Head -n**, specifies how many lines to show

tail

- Exact same as head, but outputs the **last N** lines of file

tr command

- **Translates** or **deletes** characters
- Reads from stdin only (so must use I/O operators)

Character mapping rules

- **tr 'abcde' '123'**
 - **a→1, b→2, c→3, d→3, e→3**
- Replace group (second string) maps index to index with capture group (first string)
- If replace group is one character, everything in capture group is replaced by that character.

tr command (cont.)

- **-d** deletes specified characters
 - `tr -d '|' < data.txt ->` Deletes all pipes.
- **-s** squeezes repeated characters into one
 - `tr -s ' ' < data.txt ->` Turns all amount of continuous spaces, into one space
 - `' ' -> ''`
- **-c** complements the capture group — matches everything not in the set
 - `tr -cd '0-9' < data.txt ->` Delete everything that's not a digit

sed command

- **Transform text line by line.**
 - In contrast to tr, it replaces strings with strings, rather than characters with characters.
- **Substitution command**
 - **sed -E s/<pattern>/<replacement>/** — finds <pattern> and replaces with <replacement>
 - By default, only replaces first match per line. To ensure it replaces every match in file, do:
 - **sed -E s/<pattern>/<replacement>/g** -> Add the 'global' command at the end
- **Delete command**
 - **sed -E /pattern/d** — deletes every line that matches the pattern

sed command (cont.)

- By default, sed will always output the contents of the file after transformation.
- Similar to how we teach grep, we recommend using **-E** any time you use sed for access to extended regular expressions.
- **Print command**
 - /pattern/p — prints matching lines (usually paired with -n to suppress default output)
 - Without -n, matching lines print twice (because of the first dot point)

wc (Word Count)

- Word count — counts lines, words, and characters in a file
 - `wc -l` counts lines
 - `wc -w` counts words
 - `wc -c` counts characters (bytes)
- By default, wc will print **lines words characters** in that order

UNIX Pipelines

When you grep grep's manual:

```
↳ grep --help | grep "invert"
```

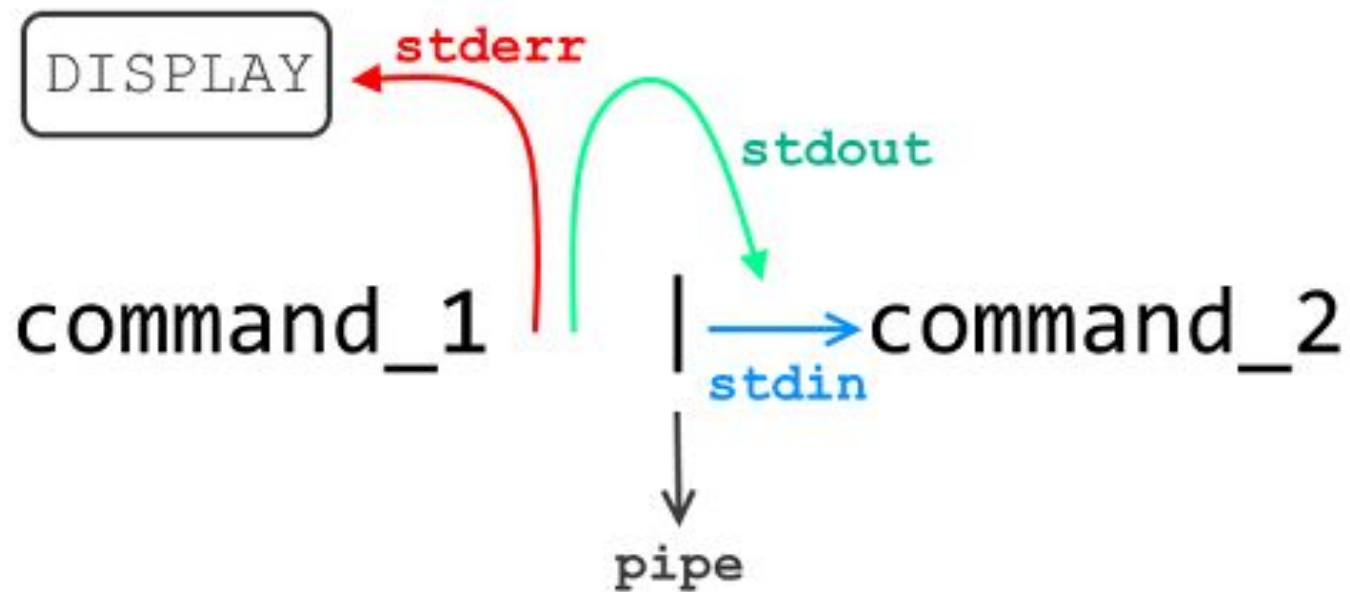


I used the grep to grep grep

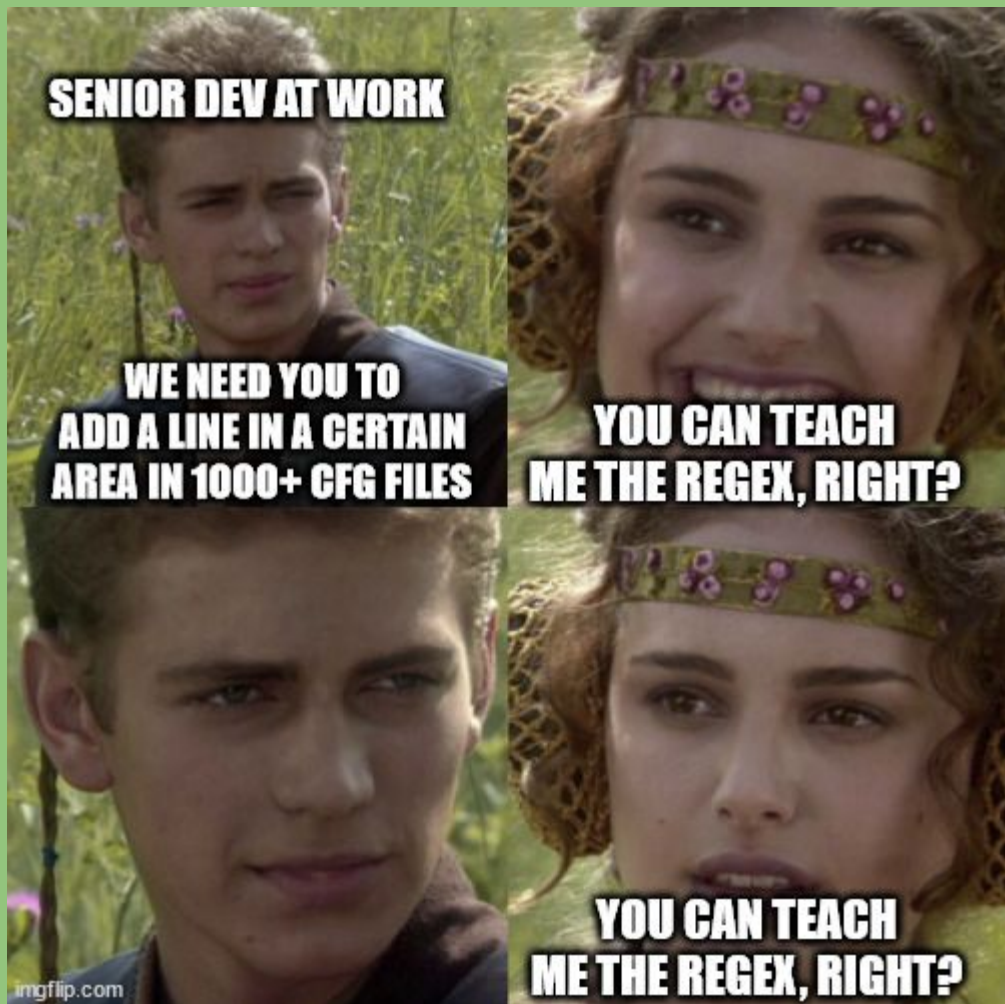
UNIX Pipelines

- Output of command passed to the next command
 - No need to save the output to a file
- Chained together to fulfil a much more complex process
- Data Parsers in their simplest form work in the same way
 - Improvements such as parallelism, clustering etc. are added on top

UNIX Pipelines



Tutorial Questions



To The Lab!

